

## A Technical Introduction to Gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

### Section 1

reformulated as a quadratic minimization problem. If the system matrix  $\mathbf{A}$  is real symmetric and positive-definite, an objective function is defined as the quadratic function, with minimization of

### Section 2

The idea is to take repeated steps in the opposite direction of the gradient (or approximate gradient) of the function at the current point, because this is the direction of steepest descent. Conversely, stepping in the direction of the gradient will lead to a trajectory that maximizes that function; the procedure is then known as gradient ascent. Gradient descent should not be confused with local search algorithms, although both are iterative methods for optimization. Gradient descent is particularly useful in machine learning and artificial intelligence for minimizing the cost or loss function. Gradient descent is generally attributed to Augustin-Louis Cauchy, who first suggested it in 1847. Jacques Hadamard independently proposed a similar method in 1907. Its convergence properties for non-linear optimization problems were first studied by Haskell Curry in 1944, with the method becoming increasingly well-studied and used in the following decades. A simple extension of gradient descent, stochastic gradient descent, serves as the most basic algorithm used for training most deep networks today.

### Section 3

Extensions Gradient descent can be extended to handle constraints by including a projection onto the set of constraints. This method is only feasible when the projection is efficiently computable on a computer. Under suitable assumptions, this method converges. This method is a specific case of the forward-backward algorithm for monotone inclusions (which includes convex programming and variational inequalities). Gradient descent is a special case of mirror descent using the squared Euclidean distance as the given Bregman divergence.

### Section 4

$J_G(\mathbf{x}) = [3 \sin(\mathbf{x}_2 \times \mathbf{x}_3) \times \mathbf{x}_3 \sin(\mathbf{x}_2 \times \mathbf{x}_3) \times 2.8 \times \mathbf{x}_1 - 1250 \times \mathbf{x}_2 + 2.0 - \mathbf{x}_2 \exp(-\mathbf{x}_1 \times \mathbf{x}_2) - \mathbf{x}_1 \exp(-\mathbf{x}_1 \times \mathbf{x}_2)] \times 20$ .  
 $3 \sin(\mathbf{x}_2 \times \mathbf{x}_3) \times \mathbf{x}_3 \sin(\mathbf{x}_2 \times \mathbf{x}_3) \times 2.8 \times \mathbf{x}_1 - 1250 \times \mathbf{x}_2 + 2.0 - \mathbf{x}_2 \exp(-\mathbf{x}_1 \times \mathbf{x}_2) - \mathbf{x}_1 \exp(-\mathbf{x}_1 \times \mathbf{x}_2)$

$$\{(-x_1x_2)\} \&-x_1\exp(-x_1x_2)\&20\end{bmatrix}\}$$

### Section 5

Using gradient descent in C++, Boost, Ublas for linear regression Series of Khan Academy videos discusses gradient ascent Online book teaching gradient descent in deep neural network context Archived at Ghostarchive and the Wayback Machine: "Gradient Descent, How Neural Networks Learn". 3Blue1Brown. October 16, 2017 – via YouTube. Garrigos, Guillaume; Gower, Robert M. (2023). "Handbook of Convergence Theorems for (Stochastic) Gradient Methods". arXiv:2301.11235 [math.OC].

### Section 6

$$\nabla f(\mathbf{x}) = 2\mathbf{A}(\mathbf{A}\mathbf{x} - \mathbf{b})$$

### Section 7

repeat in the loop:  $\mathbf{r} := \mathbf{b} - \mathbf{A}\mathbf{x}$ ;  $\mathbf{r} := \mathbf{r} / \|\mathbf{r}\|$ ;  $\mathbf{A}\mathbf{r}$ ;  $\mathbf{x} := \mathbf{x} + \eta \mathbf{r}$   
 if  $\|\mathbf{r}\|$  is sufficiently small, then exit loop  
 end repeat  
 loop  
 return  $\mathbf{x}$  as the result  
 end loop

### Section 8

Gradient descent is based on the observation that if the multi-variable function  $f(\mathbf{x})$  is defined and differentiable in a neighborhood of a point  $\mathbf{a}$ , then  $f(\mathbf{x})$  decreases fastest if one goes from  $\mathbf{a}$  in the direction of the negative gradient of  $f$  at  $\mathbf{a}$ , i.e.,  $-\nabla f(\mathbf{a})$ . It follows that, if

### Section 9

$$f(\mathbf{x}) = \mathbf{x}^T \mathbf{A} \mathbf{x} - 2 \mathbf{x}^T \mathbf{b},$$

### Section 10

$$f(\mathbf{x}_0) \geq f(\mathbf{x}_1) \geq f(\mathbf{x}_2) \geq \dots$$

### Section 11

Modifications Gradient descent can converge to a local minimum and slow down in a neighborhood of a saddle

point. Even for unconstrained quadratic minimization, gradient descent develops a zig–zag pattern of subsequent iterates as iterations progress, resulting in slow convergence. Multiple modifications of gradient descent have been proposed to address these deficiencies.

## Section 12

$$G(\mathbf{x}) = \begin{bmatrix} 3x_1 - \cos(x_2x_3) - 3x_2^4x_1^2 - 625x_2^2 + 2x_2 - 1 \exp(-x_1x_2) + 20x_3 + 10\pi - 33 \\ \frac{3x_2^2}{4x_1^2} - 625x_2^2 + 2x_2 - 1 \exp(-x_1x_2) + 20x_3 + \frac{10\pi}{3} \end{bmatrix},$$

## Section 13

This inequality implies that the amount by which we can be sure the function  $f$  is decreased depends on a trade off between the two terms in square brackets. The first term in square brackets measures the angle between the descent direction and the negative gradient. The second term measures how quickly the gradient changes along the descent direction. In principle inequality (1) could be optimized over  $\mathbf{p}_n$  and  $\eta_n$  to choose an optimal step size and direction. The problem is that evaluating the second term in square brackets requires evaluating  $\nabla f(\mathbf{a}_n - \eta_n \mathbf{p}_n)$ , and extra gradient evaluations are generally expensive and undesirable. Some ways around this problem are:

## Section 14

The line search minimization, finding the locally optimal step size  $\eta$  on every iteration, can be performed analytically for quadratic functions, and explicit formulas for the locally optimal  $\eta$  are known. For example, for real symmetric and positive-definite matrix  $\mathbf{A}$ , a simple algorithm can be as follows,